

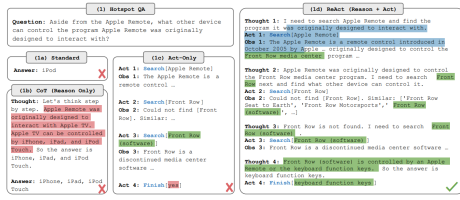
IMPERIAL

agent-spec-kit

Scenario-Driven Evaluation and Testing of Tool-Using LLM Agents

Tool-using agents act on the world

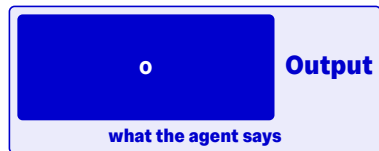
- LLMs are no longer just text generators — as **agents** they plan, call tools, and **change external state**.
- Correctness can therefore **not** be judged from the final reply alone.
- A support agent can sound perfectly polite while it:
 - skips **authentication** and leaks account data,
 - calls the **wrong tool** or in the wrong order,
 - leaves the **database** in an invalid state.
- The plausible answer is only one part of being correct.



Reasoning + acting (ReAct): the agent interleaves thoughts, tool calls, and replies.

Correctness lives on three surfaces

Output is necessary but far from sufficient



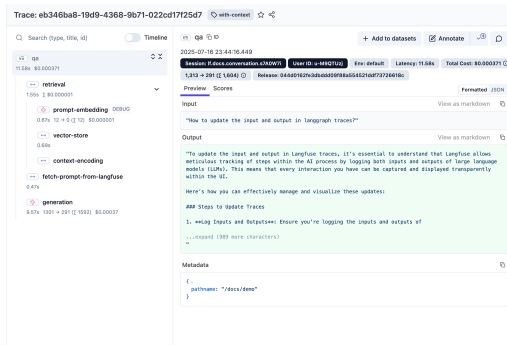
A failure on any surface can be hidden by a fluent reply — and may even be masked by recovery on a later turn.

Existing tools only cover part of the loop

- **Output graders / LLM-as-judge** (HELM, G-Eval, MT-Bench): great for comparing models on isolated final answers — but blind to tool and state failures.
- **Agent benchmarks** (τ -bench, AppWorld, WebArena): trace/state aware, but fixed leaderboards, not reusable infrastructure for your own agent.
- **Trace viewers** (LangSmith, Langfuse): show what happened — the developer still has to infer what should have happened.

The gap

No single, reusable way to assert over **output + trace + state** in one multi-turn scenario — with **precise diagnostics** when a check fails.



Trace viewers show the episode; they do not encode the expected behaviour.

Research question

Can the evaluation of **stateful, tool-using LLM agents** be expressed in a **software-testing style** —

IMPERIAL

readable scenarios that combine assertions over **replies**, **tool trajectories**, and **application state**, while preserving enough structured evidence to **diagnose** failures precisely?

Answer: agent-spec-kit — a scenario-driven testing framework for LLM agents, evaluated on a 50-task benchmark.

Part I --- The Library

agent-spec-kit: behaviour as executable scenarios

The core idea: a scenario is a conversation script

Output, tool, and state checks live together, woven in after each turn

```
@scenario(agent_fixture="agent", user_fixture="user")
async def test_billing_credit(s, store):
    (s.user_message("I was double charged in October")
     # O: the reply must insist on verifying first
     .assert_output(m.contains("verify"))
     # T: the agent must authenticate before account tools
     .assert_tool_calls([
         m.tool_call("authenticate_customer"),
         m.tool_call("get_billing", args={"id": "CUST-045"}),
     ])
     # S: a postcondition on the database / environment
     .assert_that(lambda: assert_credit_policy(store)))
```

- **O** assert_output — the reply
- **T** assert_tool_calls / forbid_tool_calls — the trace
- **S** assert_that — environment / DB state

One file reads like a dialogue with the contract inlined — no separate datasets, graders, and runner config.

One composable matcher language

- Literals, dicts and lists are coerced into matchers automatically.
- Rich combinators: `m.object`, `m.list`, `m.one_of`, `m.optional`, `m.tool_call(..., children=[...])`.
- Conditional rules:
`m.require("pager").when(m.field("severity") == "high")`.
- Selective **LLM rubrics** (`m.llm_criteria`) only where nuance is needed — pass/fail criteria, not opaque scores.

[FIGURE PLACEHOLDER]

Matcher cheat-sheet / nested `m.tool_call` example.
(Render a clean code card or reuse a report listing.)

Precise diagnostics: structured counterexamples

When a check fails, you learn which expectation broke, where

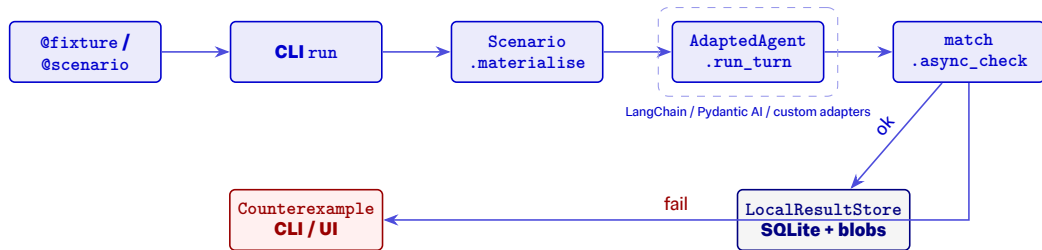
- Every failed assertion produces a **Counterexample**:
 - headline + human location
("assert_tool_calls after turn #2")
 - a **JSON-pointer path** to the mismatch, e.g. `$[1].args.line_id`
 - expected vs. actual, plus a bounded event trace
- Identical view in the **terminal**, the **SQLite** store, and the **web UI**.
- Turns “it failed somehow” into “this check broke here”.

```
progress: test_fail_arg_value [1/1]: queued
progress: test_fail_arg_value [1/1]: repeat started
FAIL test_fail_arg_value [1/1] 0.00s - assert_tool_calls: test_fail_arg_value: list item at index 1 did not match
the expected matcher for that position; re... FAIL test_fail_arg_value [1/1]

Check: assert_tool_calls
Where: 1st assert_tool_calls after turn #2 (agentturn) (tool call list for that turn)
Path: $[1].args.line_id
Expected:
equality: expected 'LINE-001'
[
  [0] tool_call("authenticate_customer"),
  [1] tool_call("order_replacement_sim", args=object({'line_id': 'LINE-001', ...}))
]
Actual:
[
  {
    "name": "authenticate_customer",
    "args": {
      "phone": "555-0100"
    },
    "result": "ok",
    "error": null,
    "children": [],
    "metadata": {}
  },
  {
    "name": "order_replacement_sim",
    "args": {
      "line_id": "LINE-002",
      "address_id": "ADDR-9"
    },
    "result": "ok",
    "error": null,
    "children": [],
    "metadata": {}
  }
]
list item at index 1 did not match the expected matcher for that position; representative mismatch at expected
index 1 (actual index 1): value does not equal expected
value does not equal expected
(+1 more matcher error(s))

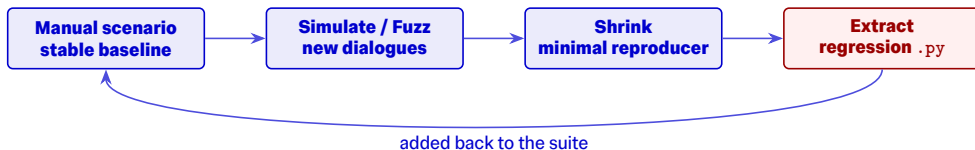
Event trace
└─ UserTurn: trace:auth-order
  └─ AgentTurn: completed 2 tool(s)
    └─ authenticate_customer result=ok
      └─ order_replacement_sim result=ok
```

Framework-neutral execution model



- Adapters **normalise traces** (LangGraph, Pydantic AI) into one event model — oracles never depend on a framework's message format.

From authored tests to automatic discovery



- **Repeats** for stochastic agents.
- **User simulation:** persona-driven multi-turn dialogues.
- **Mutation fuzzing:** perturb valid transcripts.

- **Shrinking:** delta-debug to a minimal failing dialogue.
- **Extraction:** pin a discovered failure as a permanent scenario.
- All under the same oracle vocabulary.

Persistent runs, CLI, and a web results browser

- Every run stored locally (SQLite + gzip blobs) with git metadata.
- CLI: run, runs, show, regressions, ui.
- Read-only **web UI**: runs list, run detail, trace drawer, fuzz trials.
- **Compare** named experiments side-by-side to track regressions.

TRIALS	SCENARIO	TAGS	REPEAT	STATUS	DURATION	FAILURE KIND	FAILURE MESSAGE	ASSERTIONS
—	test_007_c29_full	TELECOM FAULT-DETECTION FAULT7FEE TASK7F3 ORACLE.F MULTIANTFAULT_WRONG_CLARIFICATION	Q/1	failed	10.79s	forbid_tool_calls	forbid_tool_calls: test_007_c29_full: forbidtool tool call 'order_replacement_tool' found at index 0 (partial lo...	Q/1
—	test_007_c29_output	TELECOM FAULT-DETECTION FAULT7FEE TASK7F3 ORACLE.D MULTIANTFAULT_WRONG_CLARIFICATION	1/1	passed	51.09s	—	—	Q/0
—	test_007_c29_state	TELECOM FAULT-DETECTION FAULT7FEE TASK7F3 ORACLE.S MULTIANTFAULT_WRONG_CLARIFICATION	1/1	passed	33.97s	—	—	Q/0
—	test_007_c29_trace	TELECOM FAULT-DETECTION FAULT7FEE TASK7F3 ORACLE.T MULTIANTFAULT_WRONG_CLARIFICATION	Q/1	failed	10.12s	forbid_tool_calls	forbid_tool_calls: test_007_c29_trace: forbidtool tool call 'order_replacement_tool' found at index 0 (partial lo...	Q/1
—	test_005_340_full	TELECOM FAULT-DETECTION FAULT7FEE TASK7F3 ORACLE.F MULTIANTFAULT_WRONG_NESTED_TOOL	Q/1	failed	1m 14s	assert_tool_calls	assert_tool_calls: test_005_340_full: could not match expected element at index 1 while preserving order...	Q/1
—	test_005_340_output	TELECOM FAULT-DETECTION FAULT7FEE TASK7F3 ORACLE.D MULTIANTFAULT_WRONG_NESTED_TOOL	1/1	passed	1m 37s	—	—	Q/0
—	test_005_340_state	TELECOM FAULT-DETECTION FAULT7FEE TASK7F3 ORACLE.S MULTIANTFAULT_WRONG_NESTED_TOOL	1/1	passed	1m 2s	—	—	Q/0
—	test_005_340_trace	TELECOM FAULT-DETECTION FAULT7FEE TASK7F3 ORACLE.T	Q/1	failed	1m 7s	assert_tool_calls	assert_tool_calls: test_005_340_trace: could not match expected element at index 1 while preserving...	Q/1

Run summary

SCENARIOS 65 / 152 passed
REPEATS 65 / 152 passed
REPEAT PASS RATE 42.8%
FLAKY SCENARIOS 0
MIN REPEAT 1 / 1
56.13s 53.88s / 1m 58s
STARTED 26/05/2026, 16:29:13
FINISHED 26/05/2026, 16:48:15

Run metadata

Run ID run_20260526_152913_38e403
Experiment default
Python 3.14.0
Package 6.1.0
Git fault-seeding-calibration @ 7948c5 (d87c)
Command agent-spec-kit run /Users/ruthe/likewise/Documentation/for_testing_py/object/agent_spec_kit/benchmark/suite/trace_support/tasks/Fault_detect/sas/generator --tags Fault-detection -v 8

Failure kinds

assert_tool_calls x24
assert_tool_calls x18
assert_output x7
agent_error x10
forbid_tool_calls x5

Tag breakdown

Run detail page: per-scenario, per-repeat results with a trace drawer.

Live Demo

Writing a scenario, watching it fail precisely,
discovering a bug, and pinning it as a regression.

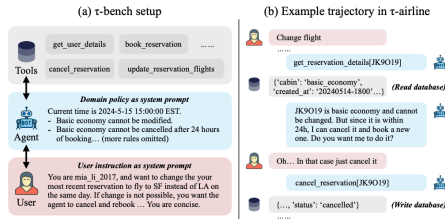
($\approx$ 8--10 minutes --- see speaker plan for the script)

Part II --- Evaluation

Does multi-surface, scenario-driven testing actually pay off?

TelcoSupportBench: the evaluation testbed

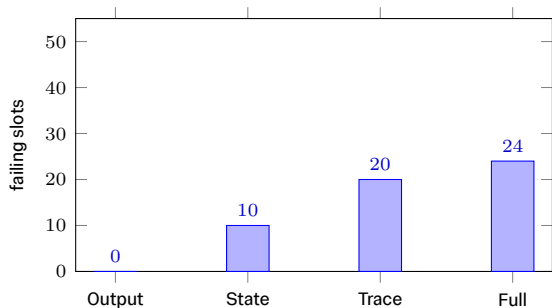
- A synthetic **mobile-support** domain built for this project.
- **50 tasks** (T01–T50) against a reference **LangGraph** agent: a coordinator + diagnostics + billing specialists.
- Each task has **four oracle lanes**: **O** output, **T** trace, **S** state, and **F** = full (all three).
- **Design principle**: tools stay permissive; **policy is asserted in the oracles**, not hidden in the simulator.
- Compared against **7** other evaluation approaches.



Inspiration: τ -bench-style dual-control support setting. [\[Replace with TelcoSupportBench architecture figure if available\]](#)

Finding 1 --- output-only grading is not enough

Reference agent across 200 oracle slots (50 tasks $\times$ 4 lanes)



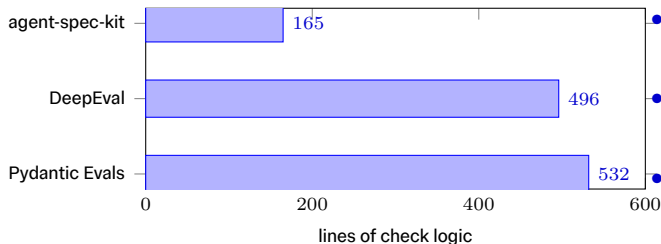
- **All 50 output lanes passed** — yet **54 of 200** slots failed.
- Every one of those failures lived in the **trace**, **state**, or **full** lanes.
- The reply “sounded correct” while a tool ran in the wrong order, an argument was wrong, or a store update never happened.

Takeaway

Final-answer evaluation alone would have reported **100%** pass.

Finding 2 --- more expressive, less code

Twelve canonical check patterns, ported across frameworks



- **165 lines** vs. 496 (DeepEval) and 532 (Pydantic Evals).
- **Highest clarity grade on every one** of the twelve specimens.
- On failures, it gives an exact **path witness** where alternatives return a scalar, broad prose, or a bare `AssertionError`.

Declarative DSL — equivalent in policy to the hand-written ports, but far easier to read and debug.

Finding 3 --- full oracles catch faults narrow lanes miss

Injected-fault detection (F01--F10) on previously-passing slots

- Ten fault families injected into the agent; measure which lane detects each.
- **Output** detection is frequently **0%** for tool-ordering and state-mutation faults.
- **State** and **Full** lanes reach **75–100%** on those same faults.
- The **combined oracle** is stronger than any single surface alone.

Fault	O	S	T	F
F02 (state)	0%	100%	67%	100%
F03 (state)	0%	100%	0%	100%
F05 (order)	0%	0%	100%	100%
F07 (trace)	50%	50%	100%	100%
F08 (trace)	0%	0%	75%	50%
F10 (full)	25%	0%	33%	100%

Detection rate = detected / eligible. Selected rows; full matrix F01–F10 in the report.

Finding 4 --- generative testing finds bugs nobody scripted

Method	Convs	Tool paths	Failure sigs
Manual scripts	14	10	3
User simulation	70	58	24
Mutation fuzz	70	36	18

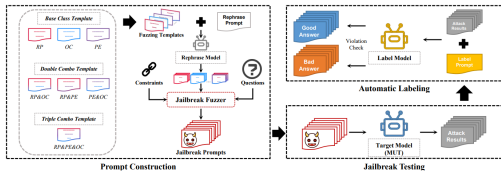
14 tasks. Simulation & fuzzing reuse the same F-oracle as the manual scripts.

- Simulation & fuzzing reached **many failure signatures absent** from the hand-written scripts — at little extra authoring cost.

Real bug found by fuzzing — T45

The `drop_required_fact` mutation garbles the opening turn (still mentions CUST-045). The agent then **names and anchors the account in prose before authentication** — a pre-auth **privacy** violation the manual script never triggered. Caught by the output rubric (2/4 criteria, needed 3).

Extraction then pins it as a permanent regression scenario.



Mutation-fuzz pipeline over valid transcripts.

Honest about the limits

Threats to validity

- **Construct:** clarity & diagnostic grades used **LLM judges**, not a human developer study.
- **External:** a **single** reference agent and **one** telecom domain — design choices may favour the framework.
- **Rerun:** extraction produced a valid file for every eligible failure, but **fewer than half** reproduced the exact signature on immediate rerun — stochastic ordering/wording drifts.
- Results are **promising** but describe this setup, not agents in general.

Conclusion

- The right unit of evaluation is the **behavioural episode**, not the final reply — a polite answer can hide a skipped auth step, a mis-ordered tool, or a corrupted store.
- agent-spec-kit unifies **output + trace + state** oracles in one readable, executable, framework-neutral scenario.
- **Diagnostics are part of evaluation quality**: assertion-local counterexamples make failures fixable, not just countable.
- A single workflow spans **discovery** → **regression**: simulate, fuzz, shrink, extract.
- Evidence: hidden trace/state failures exposed, **3× less** check code with the highest clarity, and generative testing surfacing real bugs — e.g. a pre-auth privacy leak.

Future work

- More **domains and agent frameworks** — test whether the scenario model transfers beyond TelcoSupportBench.
- **Human studies** of authoring clarity and diagnostic usefulness, to replace LLM-judge grading.
- **Signature-stable replay** after extraction, and more robust matching of stochastic trajectories.
- **Larger, team-based suites** to see how the workflow scales in everyday development.

IMPERIAL

Thank you.
Questions?

agent-spec-kit
June 2026